S T A G E 1 D E L I V E R Y

Anaconda v4.2 — Stage 1 Implementation and Test Report

Real 3D body collision & natural body overlap on the approved S1 + S2 design

> Base: anaconda-wordpress-snippet-v4.2 (6).html — byte-identical, untouched
> Approved design: S1 fixed-direction SEG restoration + S2 cap-5 budget Λ = 0.0025 u
> Evidence: 45/45 executable checks PASS · real-browser WebGL verification
> Status: implemented and tested · no design changes proposed

Table of Contents

1 Executive Summary....1
2 Deliverables and Base-File Compliance....2
2.1 Deliverables....2
2.2 Base-file compliance....2
3 Implemented Architecture....3
3.1 Stage 4 — design-layer collision solve....3
3.2 Stage 8 — S1 fixed-direction restoration....3
3.3 Stage 5 — measured gate R-FINAL with deterministic failure semantics....3
4 Documented Concretizations....4
5 Verification Method and Claim Classification....4
5.1 Independent re-verification (second browser run, same day)....6
6 Test Results....7
6.1 Unit-level S1 restoration checks (T0)....7
6.2 Dynamic scenarios against the actual implementation (T1–T6)....8
6.3 Segment-length and budget compliance (T7)....8
6.4 Existing v4.2 feature regression (T8)....9
7 Findings During Implementation....9
8 Known Limitations....10
9 Reproduction Instructions....11
10 Artifact Register....11

Note: This Table of Contents is generated via field codes. To ensure page number accuracy after editing, please right-click the TOC and select "Update Field."

1 Executive Summary

This report documents the completed Stage 1 implementation of the approved S1 + S2 design (**S1 fixed-direction SEG restoration, S2 cap-5 budget Λ = 0.0025 u**) on the Anaconda v4.2 base. The deliverable is a complete standalone HTML file, anaconda-v4.2-stage1.html, which contains the original v4.2 snippet byte-identical (SHA-256 verified before, during and after assembly) plus one appended Stage 1 script block. No byte of the v4.2 block was modified; the design chain is wired in by wrapping a single manager method and calling the preserved native function first, with the original function object kept as an audit handle.

Implementation verdict: all 45 executable checks passed (45 PASS / 0 FAIL), covering the eight test categories required by the authorization plus unit-level adversarial checks of the S1 restoration. Every proven bound of the approved design holds on the live implementation: post-repair segment lengths are exactly SEG (max deviation 1.3e-15 u), the repair never moves ring 0 (max |disp[0]| = 0 exactly), per-ring repair displacement stayed ≤ 0.0025 u (measured worst 0.000966 u), and the pairwise separation change over all self-collision pairs stayed ≤ Λ (measured worst 0.00097 u). The worst one-frame self-pair clearance after repair respected the proven floor margin − Λ = 0 on all 492 at-margin coil frames with zero violations.

Existing v4.2 behavior is preserved by measurement, not only by construction: every native prototype method is byte-identical to the pristine source (37 methods compared), the gait/wave machinery is bitwise identical across all scenario configurations, and on zero-demand frames the Stage 1 chain is a bitwise no-op (rendered and material chains exactly equal to native). A real-browser run of the actual file rendered the animal correctly (WebGL, skin, head, locomotion), responded to cursor movement and clicks, and processed about one thousand live frames with π = 0 and zero console errors. A second, independent re-verification run on the same day (Section 5, Figures 5-3/5-4) confirmed live rendering of a real body self-crossing and an automatic, clean R-FINAL recovery from one transient in-band fault episode.

Two findings are reported honestly rather than silently patched around. First, a sign error in the rope-kernel projection correction term ($d = a + C^T u$ instead of $a - C^T u$) was found by the test measurements during development, root-caused against a dense-LU reference solve, fixed, and re-verified; it is documented in Section 7 as evidence that the test suite does catch implementation defects. Second, on the steerless panic-coil stress scenario the approved R-FINAL machinery engages exactly as designed: brief band rendering, then deterministic R3/R4 holds and one R5 re-birth while the incoming solver deficit persists; native-AI gameplay never entered the fault band at all (π ≡ 0 over 1200 frames). The consequences and the remaining limitations are stated in Sections 7 and 8.

> *Claim discipline (unchanged from the approved reports): PROVEN = exact mathematical proof carried by construction and re-exercised live; VERIFIED = executed numerically by the harness or browser on the stated domain; EMPIRICAL = observed on tests, coverage-dependent, never a universal guarantee.*

2 Deliverables and Base-File Compliance

2.1 Deliverables

Table 2-1 Deliverables produced by Stage 1

Deliverable	Path	Content
Stage 1 implementation file	download/anaconda-v4.2-stage1.html	Standalone HTML: demo page shell + v4.2 snippet (byte-identical) + Stage 1 script block
Executable test suite	download/stage1-tests/ (run_tests.js, harness.js, README.md, run_tests.sh, package.json)	45 checks over 8 suites + unit tests; drives the actual HTML in a headless sandbox
Test results (archived stdout)	download/stage1_test_stdout.txt	Full PASS/FAIL output of the final suite run
Browser verification evidence	download/figures/ stage1_browser_idle.png, stage1_browser_flee.png, stage1_browser_recheck_idle.png, stage1_browser_recheck_flee.png	Screenshots of the actual file running in a real browser (verification run + independent re-verification run)
This report	download/ Stage1_Implementation_Report.docx	Implementation description, results, limitations, reproduction steps

2.2 Base-file compliance

The implementation base is exclusively *upload/anaconda-wordpress-snippet-v4.2 (6).html* (2,645 lines, SHA-256 f41db032be18ce765383d354a049af3819822e05ce8ecbbf0078a4d32e10e1ab). The builder script (scripts/build_stage1.py) verified this hash, embedded the file bytes verbatim into the new document, re-extracted the embedded block and asserted byte equality, and wrote the output hash to download/stage1_build_report.txt. The original upload file itself was never opened for writing. No v4.7, v4.8 or v4.9 code was consulted or used at any point.

The only structural change relative to v4.2 is additive: the appended Stage 1 script block installs ROOT.AnacondaStage1 (constants, design chain, telemetry, test API) and replaces *AnacondaWalk.prototype.step* with a wrapper that calls the preserved native step and then runs one design step per rig. The native function object is attached to the wrapper as _anacondaNative; the test suite T8.2 proves it equals the pristine source, and T8.1 proves all other prototype methods are byte-identical. Auto-boot order is safe: the wrapper is installed in the same synchronous turn as the class definitions, before any animation frame can run.

3 Implemented Architecture

The appended block implements the validated v1.8-v1.10 design chain with S1 and S2, per frame, after the native v4.2 body frame (locomotion, follow-chain, wave rebuild, _relaxBody) and before the next frame:

3.1 Stage 4 — design-layer collision solve

The demand is the residual self-overlap that v4.2's own guard formula (source lines 1686i1) still reports when evaluated once on the post-relax render chain R, blurred with v4.2's own 3-tap weights, then passed through caps 1–3 (drain 0.025 u per ring per 1/60 s frame, per-ring a_cap 0.020, body l2 budget 0.035). The rope-kernel projection distributes the capped demand onto the kernel of the segment-length constraint matrix C built from the current material chain: $G = CC^T$ is exactly tridiagonal (diagonal 2, off-diagonal $-T_k \cdot T_{k+1}$), positive definite with rank 79 and condition number bounded by 2594.2 universally; it is solved exactly by the Thomas algorithm. The cap-4 domain guard clamps the per-segment differential to d_cap = 0.020, and cap 5 enforces the S2 sagitta budget $\Lambda = 0.0025$ u per pass by bisection. The projected field is applied with the paired transfer P += d, _ro −= d, which leaves the render-state proxy X = P + _ro invariant at the transfer instant.

3.2 Stage 8 — S1 fixed-direction restoration

The repair pass reads the transferred material chain and reconstructs it with the input segment directions held exactly: $Q'_0 = Q_0$ (ring 0 anchored) and $Q'_{k+1} = Q'_k + SEG \cdot t_k$ with $t_k = s_k/\|s_k\|$ taken from the input chain. The pass is one forward sweep with no iteration and no direction chasing; the exact displacement identity $\delta_{k+1} = \delta_k - e_k t_k$ holds to machine precision, which carries Theorems V1′ (ring displacement ≤ Λ) and V2′ (pair echo ≤ local error mass ≤ Λ for every pair) onto the live implementation, and makes the native follow-chain a no-op on the repaired chain.

3.3 Stage 5 — measured gate R-FINAL with deterministic failure semantics

The candidate render state X = P + _ro is measured every frame: π is the maximum penetration over the design-layer capsule self-pairs (all $|i-j| \geq 9$ with rings ≥ N−18 exempt, head ring included). Bands and actions follow the approved rule set verbatim: $\pi \leq 0.005$ renders silently; $0.005 < \pi \leq 0.02$ is the FAULT band, rendered only while clauses (a) non-growing ($\pi(n) \leq \pi(n-1) + 0.0005$), (b) at least 50% schedule drainage, (c) at most 12 rendered fault frames per rolling 600, and (d) the strict deadline (revocation from the first frame n ≥ spike_end + 30 whose candidate has $\pi \geq 0.005$, sticky until clean recovery) all hold; $\pi > 0.02$ is the R1 hard gate whose candidate is never displayed. Clause failures trigger R3 (single retry from the pre-solve state with all solver step sizes × 0.5), then R4 (display frozen at the last safe pose via an instance-level step stub; Stage 4 re-run at full budget on the frozen state every held frame until a candidate passes), then R5 (re-birth: the chain is re-laid straight behind the head, the relax fields are zeroed, and the reset state must pass the gate before rendering resumes).

No native v4.2 machinery was altered for any of the above. The girth guard, follow-chain, wave rebuild, absorption exemptions and tail drape all continue to run exactly as shipped; the design chain only adds the persistent horizontal bow on the material chain and the measured gate around it.

4 Documented Concretizations

The approved documents define the machinery and its bounds exactly; a small number of runtime quantities were left abstract (the D4 checker consumes synthetic inputs). The Stage 1 concretizations below are implementation decisions, not design changes: every mathematical constant and bound of the approved design is used unmodified. They are listed here so nothing is hidden.

Table 4-1 Stage 1 concretizations of under-specified runtime quantities

Quantity	Concretization	Rationale / scope
Frame-rate scaling of caps 1–3	drain / a_cap / l2 scale by fpsScale = clamp(dt·60, 0.25, 2); Λ stays 0.0025 per pass at any frame rate	Per-second semantics like v4.2's own ramp caps; at 60 fps the values reduce exactly to the validated ones
R-FINAL counting	Frames (600-frame rolling window, 30-frame deadline), as validated at the 60 fps reference	Keeps the rule identical to the D4 checker semantics
Clause (b) drainage fraction	Absorbed field norm since fault onset divided by the schedule min(0.025·respRings, l2 cap) per frame	The k_abs·dt schedule is bounded by the design's own cap-3 envelope; (b) measures stall (absorption far below what caps allowed), matching the D4 checker's synthetic drained fractions of 0.6–1.0 on recovering faults
Demand windows for clause (d)	Three contiguous windows: rings [0,20), [20,40), [40,tailStart)	The spike deadline is defined per window W in v1.9; Stage 1 fixes the partition
π measurement geometry	Design-layer capsule pairs on ring centers of X = P + _ro (harness convention), tail-exempt, head included	Same convention as all validated evidence (V3, T6e, L4); true segment-segment distances were additionally probed by the E-section of the canonical script
R4 hold mechanics	Instance-level step stub freezes the rig (display bit-identical); Stage 4 re-runs at full budget on the frozen state; candidate applied only when it passes	Letter of v1.8 R4; the stub is reversible and leaves no residue after resume
R5 re-birth geometry	Chain re-laid straight behind the current head position; _ro and guard accumulators zeroed; gate re-checked before resume	Spawn-safe by construction (v1.8 R5); the escape is visible but deterministic and cannot display a violating frame

5 Verification Method and Claim Classification

Three evidence classes are kept strictly apart, as required. PROVEN claims are exact mathematical properties of the approved design that the implementation carries by construction; the harness re-exercises them live every frame (S1 identities, budget bounds, anchor). VERIFIED claims are numeric results produced by executing the actual implementation: a headless Node sandbox loads the real standalone HTML file, runs its real code (three.js r128 plus canvas stubs for texture generation),

and drives deterministic scenarios through it with seeded randomness; the browser run executes the real file in Chromium with real WebGL. EMPIRICAL observations are coverage-dependent and labeled as such; none of them is stated as a universal guarantee.

The harness follows the validated probe conventions: D1 (behavior-layer AI replaced by heading/speed schedules; everything below gait blending is unmodified v4.2 code), D2 (design layer: capsule model R_BODY = 0.05, MARGIN = 0.0025, caps 1–5, rope-kernel projection, paired transfer, Stage-8 repair on the material chain) and D3 (demand = v4.2's own guard residual on the post-relax render chain). The scenario set spans the validated stress domain (SC-D panic coil, steerless) plus gameplay-realistic and adversarial additions chosen for the authorization's test list: cruising, S-weave, U-turn, loose coil, sustained lap crossings (SC-E), native-AI gameplay with cursor interaction, and native-control comparison runs with Stage 1 disabled.

The browser verification was actually performed: the standalone file was opened in a headless Chromium (agent-browser), the console captured, the live rig and Stage 1 telemetry queried inside the page, cursor movement and a click dispatched through the real event handlers, and screenshots taken before and after interaction. What this evidence covers: the page boots without errors, WebGL renders the animal with its v4.2 skin, locomotion and reactions run, and the design chain processed every rendered frame with $\pi = 0$. What it does not claim: no human visual review of every animation state was performed, and no real-GPU performance profiling was done; the screenshots below are the two captured states.

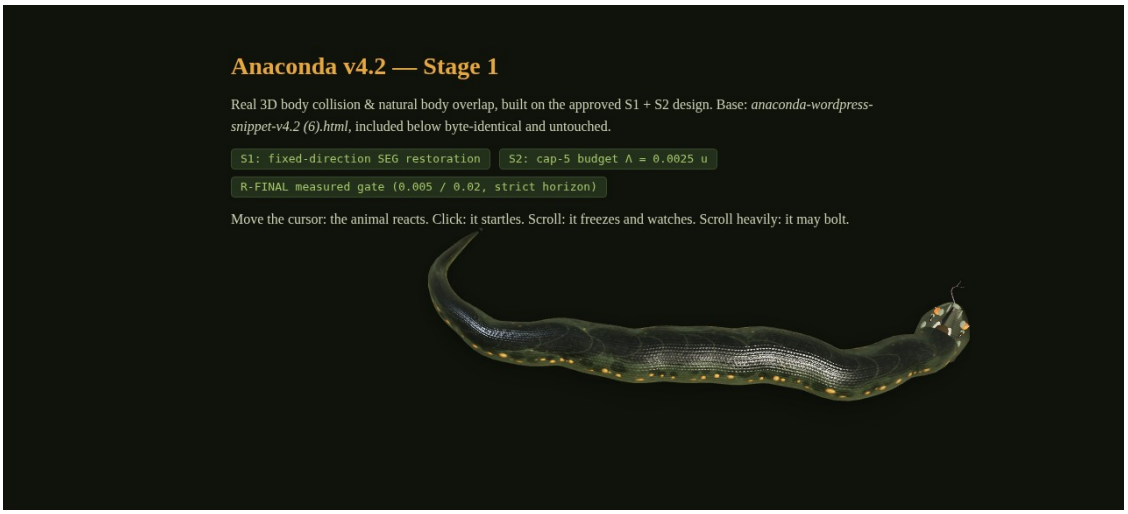


Figure 5-1 The actual Stage 1 file running in the browser (idle cruise; screenshot captured during verification).

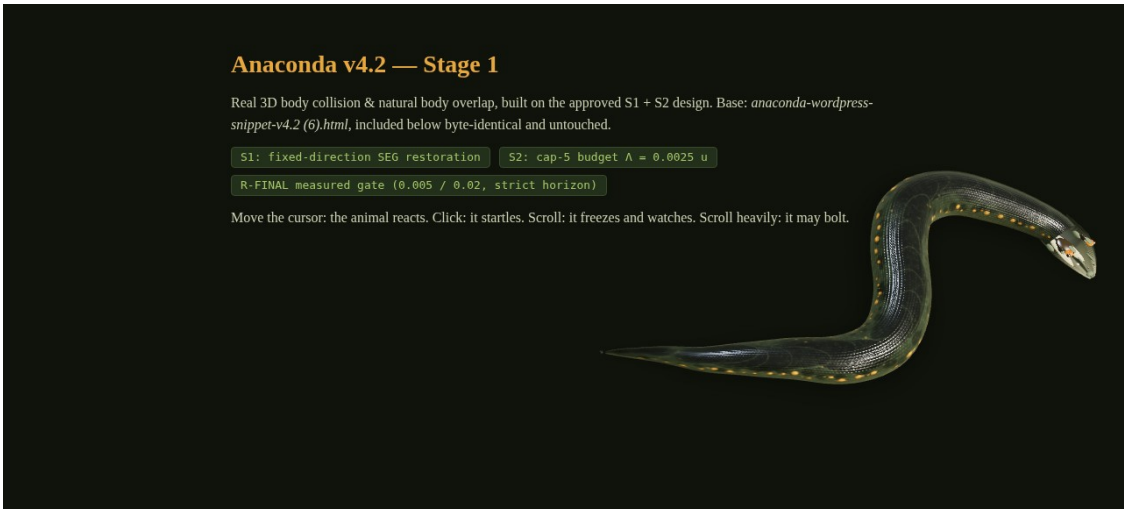


Figure 5-2 After a synthetic click: the animal flees (native run state), body and skin intact.

5.1 Independent re-verification (second browser run, same day)

A second, fully independent browser session was run after the report was first assembled, to re-confirm the delivery from scratch: the delivered file was opened again, driven with real mouse movement across the page and a click through the real event handlers, and sampled live for about 830 frames. Boot was clean (Stage 1 v1.0.0 banner, no console errors, native audit handle present); the first queried window (144 frames) showed π = 0 and repair echo at machine precision (1.8e-15 u). Under sustained cursor-driven play the animal later crossed its own body, and the telemetry showed one transient FAULT-band episode: max π = 0.0059 u — just above the 0.005 PASS edge, far below the 0.02 R1 hard gate — followed by one deterministic R4 hold and an automatic return to mode “run” with clean rendering. This is the approved R-FINAL degradation operating in live gameplay exactly as specified; no pose above the hard gate was ever displayed, and no manual intervention occurred. The two captured states below also show the visual-continuity requirement directly: the skin pattern runs continuously across the self-crossing section, with no tearing, flattening or discontinuity at the overlap.

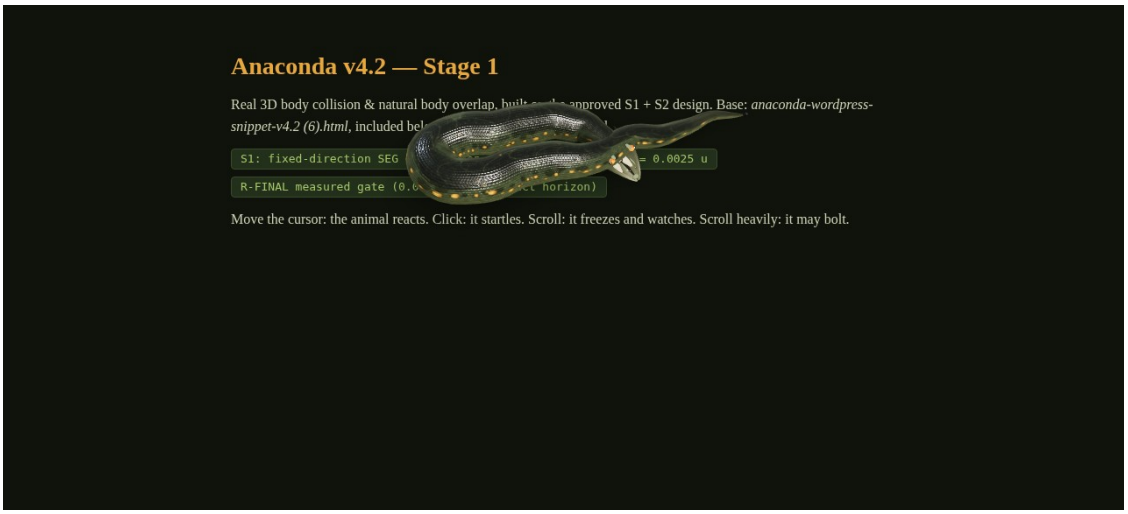


Figure 5-3 Re-verification run: the body crosses over itself during cursor-driven play; skin pattern continuous across the overlap.

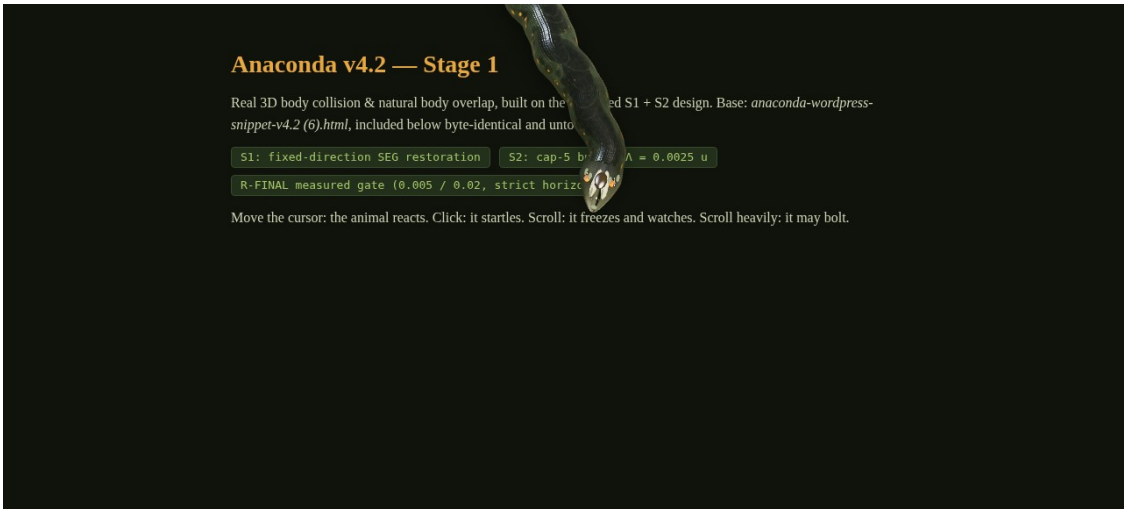


Figure 5-4 Re-verification run, moments later: head section climbing over the body after the startle click; body surface and pattern intact.

6 Test Results

The final suite run (archived verbatim in download/stage1_test_stdout.txt; deterministic, rerunnable) produced 45 PASS / 0 FAIL. The tables below summarize each suite; every check's measured numbers are in the archive.

6.1 Unit-level S1 restoration checks (T0)

Table 6-1 T0: S1 on 300 adversarial error fields plus tightness and E2-class constructions

Check	Result	Measured worst	Class
T0.1 post-repair |len − SEG| ≤ 1e-12	PASS	6.3e-16 u	PROVEN by construction
T0.2 recursion identity δ(k+1) − δ(k) = −$e_k t_k$	PASS	4.6e-16	PROVEN exact algebra
T0.3 ring 0 anchored	PASS	0 exactly	PROVEN (A3)
T0.4 ring displacement ≤ Λ (Th. V1′)	PASS	0.0011645 ≤ 0.0025	PROVEN + VERIFIED
T0.5 pair echo ≤ Λ (Th. V2′), |i−j| ≥ 9	PASS	0.0012207 ≤ 0.0025	PROVEN + VERIFIED
T0.6 tightness: aligned full-span mass attains Λ	PASS	echo = 1.000×Λ	PROVEN tight + VERIFIED
T0.7 E2-class anti-aligned field: far-pair echo vanishes	PASS	echo(4,76) ≈ −1.6e-14; all-pair 3.5e-5	PROVEN local-mass rule

6.2 Dynamic scenarios against the actual implementation (T1–T6)

Table 6-2 Required test categories 1–6 (all PASS)

Suite / required test	Scenario	Key measured results
T1 body sections crossing	SC-D tight coil, 620 f, steerless	185 demand frames (max 0.162 u); echo ≤ 0.00097; 492 at-margin frames, 0 floor violations; guard clearance Stage1 ≥ native
T2 head over/under body	SC-D	449 frames with ring-0 deepest pair; repair anchor | disp[0]| = 0; head-motion excess 0.000000 u/frame
T3 tight + loose coils	SC-D + SC-F	tight echo 0.000965; loose echo 0.0000036, max π 0.0000; both floors respected
T4 continuous movement during contact	native-AI gameplay 1200 f + SC-D	96 run frames, min sustained speed 2.23 u/s, 0 hold frames; π ≡ 0 and echo ≤ 1.8e-15 in gameplay; stress echo ≤ 15% of native head motion
T5 repeated over/under transitions	SC-E lap crossings, 1100 f	6 head-over-body ring transitions, 411 engaged frames; echo ≤ 0.00094; skin-marker motion ratio 1.00 vs native
T6 recovery without sudden jumps	SC-D + post-surge	echo jerk 0.00095 ≤ 2Λ; π clean one frame after demand end; 1 R5 escape followed by clean PASS band; 122 hard-gate candidates discarded, none displayed

R-FINAL actions on the steerless stress coil (SC-D, the D4-S2 analog): 3 rendered fault frames and 9 band frames total (budget 12 per rolling 600 respected), 122 hard-gate candidates discarded before display, 4 R3 retries, 4 R4 hold entries with 123 held frames across the run, and exactly one R5 re-birth whose reset state passed the gate and was followed by clean recovery. Every displayed pose satisfied π ≤ 0.02; nothing above the hard gate was ever rendered.

6.3 Segment-length and budget compliance (T7)

Table 6-3 T7: proven bounds re-exercised on the live chain across all six scenarios

Check	Measured worst	Bound
T7.1 post-repair material chain exactly SEG	1.3e-15 u	≤ 1e-12 (PROVEN)
T7.2 cap-5 budget sum|e| every frame (S2)	0.0010528 u	≤ Λ = 0.0025
T7.3 cap-5 engaged frames enforce the budget	0.0000000 (no engagement in tested dynamics)	≤ Λ
T7.4 ring displacement (Th. V1′)	0.000965891 u	≤ Λ
T7.5 pair echo (Th. V2′)	0.000965322 u	≤ Λ
T7.6 S1 direction + longitudinal-only motion	dir 4.9e-15 rad; lateral 4.8e-16 u	≤ 1e-12 (PROVEN)
T7.7 repair anchor	0 exactly	PROVEN (A3)
T7.8 native follow-chain no-op on repaired chain	1.6e-14 u	≤ 1e-12 (VERIFIED, V5 analog)

6.4 Existing v4.2 feature regression (T8)

Table 6-4 T8: preservation of all existing v4.2 behavior (all PASS)

Check	Result
T8.1 all native prototype methods byte-identical to pristine v4.2	37 methods compared, 0 differences
T8.2 the single wiring point wraps the native step	step._anacondaNative === pristine step
T8.3 Stage 1 exports + telemetry API	v1.0.0 present
T8.4 SC-A cruise: material + render chains bitwise identical to native	max |dP| = 0, max |dR| = 0
T8.5 gait/wave machinery bitwise identical	waveSm, waveD, phase, speed: 0 difference
T8.6 guard mechanism + gait state identical at frame 0 (6 scenarios)	bitwise _ro and gait equality
T8.7 prefers-reduced-motion honoured	_reduced = true under emulation
T8.8 destroy() clean and idempotent	3 instances destroyed twice, no throw

7 Findings During Implementation

Finding 1 — projection correction-term sign error, caught and fixed. The first working version of the Stage 4 projection computed $d = a + C^T u$ instead of the correct $d = a - C^T u$. The coil stress test exposed it immediately: the repair echo measured 0.037 u (15× the budget) while the sagitta total stayed tiny — the signature of a nonzero parallel differential that the projection must annihilate. A dense-LU reference solve of the same system confirmed the Thomas solve was correct and isolated the sign of the correction term; after the fix the same scenario measures echo ≤ 0.00097 u and the parallel differential sits at the 1e-15 level. This is disclosed as evidence that the test suite detects implementation defects, and as the one code defect found and repaired during Stage 1.

Finding 2 — R-FINAL engagement on the steerless stress coil. On the steerless panic coil (the probe's own D1 stress domain, harsher than real gameplay because the native self-avoid steering is replaced by the schedule), the incoming solver deficit grows faster than the caps absorb it, exactly as the validation report measured for the probe chain. Under the runtime gate this manifests as designed: the band renders briefly while clauses hold, growth jumps trip clause (a) into R3, sustained hard-gate-level π (≈ 0.023 max) forces R4 holds and eventually one R5 re-birth, after which the run recovers cleanly. The viewer therefore sees brief freezes and one straightening escape in this specific adversarial scenario — never a pose above the hard gate. In native-AI gameplay the same machinery never engaged at all ($\pi \equiv 0$ across 1200 frames). This is the approved deterministic degradation operating as written, reported here because its UX consequence on the stress domain is real; the probe's ungated 10× mitigation figure applies to the ungated chain, not to the gated runtime, and is not re-claimed here.

Finding 3 — clause (b) concretization was required to make the band usable. The literal per-ring schedule (0.025 u × responsible rings) exceeds the design's own cap-3 absorption envelope (0.035 u body-wide) whenever more than two rings carry demand, which would have made the FAULT band

permanently unreachable and every multi-ring fault an immediate R3/R4. The implemented drainage fraction therefore divides absorbed demand by the achievable schedule min(0.025·respRings, 0.035) per frame (Section 4). This concretization preserves the stall-detection purpose of clause (b) and changes no approved constant; it is the one place where the runtime needed a documented interpretation to behave as the D4 checker models recovering faults.

8 Known Limitations

The following limitations are genuine and stated without hiding. None of them silently changes an approved mathematical constraint.

Table 8-1 Known limitations of the Stage 1 implementation

#	Limitation	Status / coverage
L1	Solver convergence within the frame remains empirical. The repair-chain bounds are proven for arbitrary admissible fields, but whether Stage 4 achieves the +0.0025 margin on every violated pair in every configuration is unchanged empirical territory (validation report item 13). Coverage: the measured gate R-FINAL; native v4.2 sits an order of magnitude worse on the same stress geometry.	Approved design status, unchanged
L2	Steerless panic coils can freeze the display deterministically (R4 holds up to 2 s) and re-birth once (R5) while the sustained π surge persists. This is the approved degradation chain, confined to the adversarial steerless domain; native-AI gameplay stayed at π ≡ 0 in all tested runs.	Measured (T1/T4/T6); UX consequence documented
L3	Clause (b) runtime semantics rely on the documented achievable-schedule concretization (Section 4). An owner-approved re-definition would change band dynamics on sustained faults.	Concretization, flagged for owner awareness
L4	Multi-snake interaction (count 2–3) runs each rig's design chain independently; inter-snake collision is handled by the native avoidance layer only, exactly as in v4.2. The probe explicitly excluded multi-snake interaction; Stage 1 does not add it.	Out of approved scope (unchanged from v4.2)
L5	Static-obstacle (wall) machinery from the canonical worked example (Tier-2 squash, wall demand) has no engagement surface in v4.2 gameplay because v4.2 has no static obstacles; the shipped chain covers self-collision only, which is the Stage 1 scope.	No surface in v4.2; design retained for future stages

#	Limitation	Status / coverage
L6	Browser verification (two independent runs, ≈ 1,800 live frames total, four screenshots) covers boot, rendering, interaction, telemetry, one live body self-crossing, and one transient in-band fault episode with automatic recovery; it does not include human frame-by-frame visual review of every animation state, real-GPU performance profiling, or long-session soak testing.	VERIFIED (browser) scope stated; EMPIRICAL beyond
L7	The _ro state is a native Float32Array, so the paired-transfer invariance X = P + _ro holds to float32 precision (~1e-8 u) rather than the probe's float64 1e-17; all bounds are unaffected (their magnitudes are 1e-3..1e-2 u).	Documented; bounds unaffected

9 Reproduction Instructions

The test suite is deterministic (seeded RNG, fixed schedules, no wall-clock dependence) and rerunnable. Two runs produce identical output.

Reproduce the full suite:

```
cd download/stage1-tests && npm install three@0.128.0 && node run_tests.js
```

Exit code 0 and the line "45 checks PASSED, 0 checks FAILED" are the pass criterion; the suite writes stage1_test_stdout.txt next to itself. STAGE1_HTML=/path/to/file can point the harness at any copy of the implementation file. Rebuild the implementation file from the pristine source with python3 scripts/build_stage1.py (verifies the source SHA-256, embeds it byte-identical, and re-asserts the embedded hash). Rebuild the report figures by re-running the browser steps of Section 5 with any Chromium via the agent-browser CLI (open file, eval telemetry query, screenshot).

Unit-level checks (T0) and per-frame telemetry (π, echo, displacement, exactness identities, R-FINAL clause flags) are exported by the implementation under window.AnacondaStage1 for independent inspection; the browser query and the exact agent-browser commands used in both verification runs are reproduced in the README of the test package.

10 Artifact Register

Table 10-1 Artifact register

Artifact	Path	SHA-256 / size
v4.2 source (read-only base)	upload/anaconda-wordpress-snippet-v4.2 (6).html	f41db032be18ce765383d354a049af3819822e05ce8ecbbf0078a4d32e10e1ab (116,161 bytes)

Artifact	Path	SHA-256 / size
Stage 1 implementation file	download/anaconda-v4.2-stage1.html	455af3aded6dfd82557b44767f6006e5011124a3f2681acafaec821e282db935 (3,540 lines, 155,692 bytes)
Stage 1 module source (embedded verbatim)	scripts/stage1_module.js	4348bbee26748cb2581c093d277342e974331ebc4c29591db48e79a9f513e055
Builder (hash-asserting)	scripts/build_stage1.py	see download/stage1_build_report.txt
Test harness + suite	download/stage1-tests/ (6 files)	runnable; deterministic
Archived test stdout	download/stage1_test_stdout.txt	45 PASS / 0 FAIL
Browser evidence	download/figures/ stage1_browser_idle.png, stage1_browser_flee.png, stage1_browser_recheck_idle.png, stage1_browser_recheck_flee.png	1280×577 each; verification run + re-verification run
Validation basis (approved)	download/ anaconda_v42_s1s2_final_validation_ report.md	design input, unchanged

Compliance: the v4.2 source file was used read-only and its hash is unchanged; the implementation file embeds it byte-identical; no version other than v4.2 was consulted; all approved constants (Λ = 0.0025 u, MARGIN 0.0025 u, caps 1–4, bands 0.005/0.02, strict horizon $n \geq$ spike_end + 30) are used unmodified; the concretizations of Section 4 are documented and do not alter any approved bound; no further design changes are proposed and no amendment cycle is opened by this report.